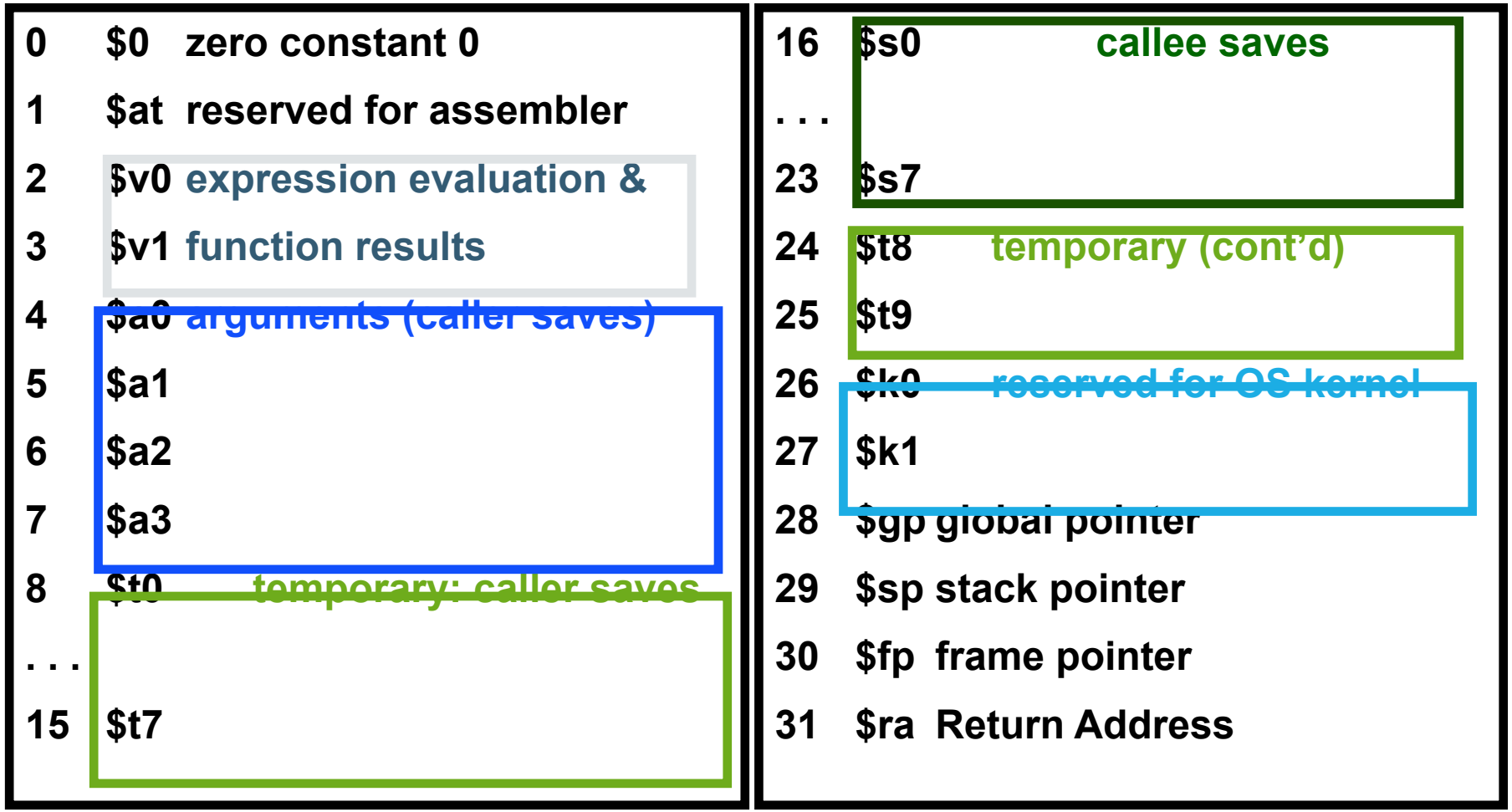
MIPS

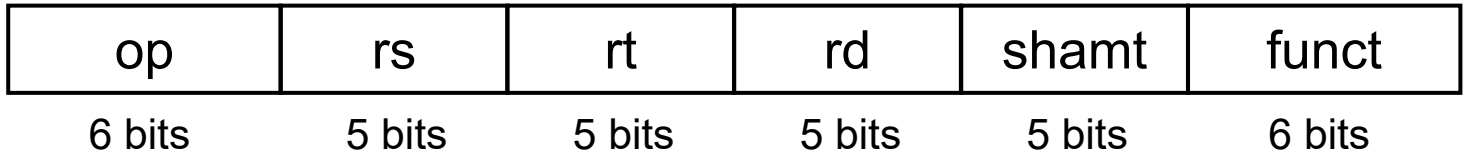
Quick Reference Guide

MIPS: Software Register Convention

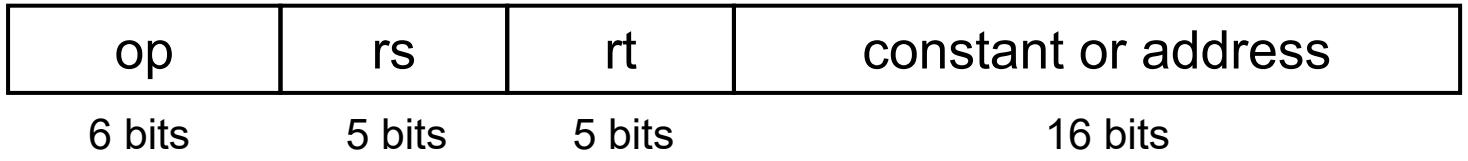


Instruction Formats

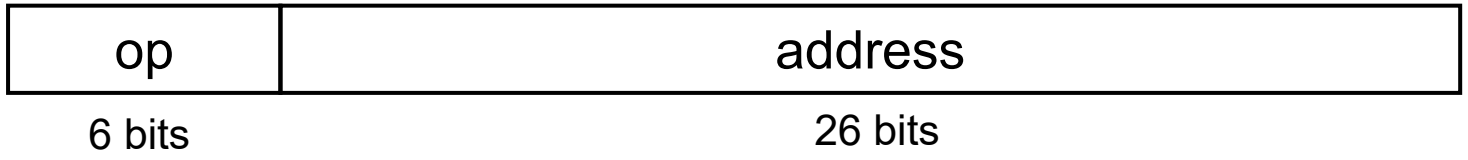
- R-type



- I-type



- J-type



Operation Codes

- Every instruction has a 6-bit opcode:
  - Bits 31-29 specify row
  - Bits 28-26 specify column

op(31:26)								
28–26 31–29	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	R-format	Bltz/gez	jump	jump & link	branch eq	branch ne	blez	bgtz
1(001)	add immediate	addiu	set less than imm.	sltiu	andi	ori	xori	load upper imm
2(010)	TLB	FlPt						
3(011)								
4(100)	load byte	load half	lwl	load word	lbu	lhu	lwr	
5(101)	store byte	store half	swl	store word			swr	
6(110)	lwc0	lwc1						
7(111)	swc0	swc1						

Function Codes

- R-type instructions have an opcode of 000000
  - Instruction is determined by funct field
  - Bits 5-3 specify row
  - Bits 2-0 specify column

op(31:26)=000000 (R-format), funct(5:0)								
2–0 5–3	0(000)	1(001)	2(010)	3(011)	4(100)	5(101)	6(110)	7(111)
0(000)	shift left logical		shift right logical	sra	sllv		srlv	srav
1(001)	jump reg.	jalr			syscall	break		
2(010)	mfhi	mthi	mflo	mtlo				
3(011)	mult	multu	div	divu				
4(100)	add	addu	subtract	subu	and	or	xor	not or (nor)
5(101)			set l.t.	sltu				
6(110)								
7(111)								

MIPS Integer Arithmetic Instructions

Instruction	Example	Meaning	Comments
add	add $1,$2,$3	$1 = $2 + $3	3 operands; exception possible
subtract	sub $1,$2,$3	$1 = $2 – $3	3 operands; exception possible
add immediate	addi $1,$2,100	$1 = $2 + 100	+ constant; exception possible
add unsigned	addu $1,$2,$3	$1 = $2 + $3	3 operands; no exceptions
subtract unsign	subu $1,$2,$3	$1 = $2 – $3	3 operands; no exceptions
add imm unsign	addiu $1,$2,100	$1 = $2 + 100	+ constant; no exceptions

Note: Immediates are sign-extended to form constant for arithmetic operations

MIPS Integer Arithmetic Instructions

Instruction	Example	Meaning	Comments
multiply	mult $2,$3	Hi, Lo = $2 x $3	64-bit signed product
multiply unsign	multu $2,$3	Hi, Lo = $2 x $3	64-bit unsigned product
divide	div $2,$3	Lo = $2 ÷ $3, Hi = $2 mod $3	Lo = quotient Hi = remainder
divide unsign	divu $2,$3	Lo = $2 ÷ $3, Hi = $2 mod $3	Unsigned quotient Unsigned remainder
Move from Hi	mfhi $1	$1 = Hi	Used to get copy of Hi
Move from Lo	mflo $1	$1 = Lo	Used to get copy of Lo

MIPS Logical Instructions

Instruction	Example	Meaning	Comment
and	and $1,$2,$3	$1 = $2 & $3	Logical AND
or	or $1,$2,$3	$1 = $2 | $3	Logical OR
nor	$1,$2,$3	$1 = ~($2 |$3)	Logical NOR
and immediate	andi $1,$2,10	$1 = $2 & 10	Logical AND w. constant
or immediate	ori $1,$2,10	$1 = $2 | 10	Logical OR w. constant
shift left log	sll $1,$2,10	$1 = $2 << 10	Shift left by constant
shift right log	srl $1,$2,10	$1 = $2 >> 10	Shift right by constant
shift left log var	sllv $1,$2,$3	$1 = $2 << $3	Shift left by variable
shift right log var	srlv $1,$2, $3	$1 = $2 >> $3	Shift right by variable

Note: Immediates are zero-extended to form constant for logical operations

MIPS Load/Store Instructions

Instruction	Example	Meaning	Comments
store word	sw $3, 8($4)	Mem[$4+8]=$3	Store word
store halfword	sh $3, 6($2)	Mem[$2+6]=$3	Stores only lower 16 bits
store byte	sb $2, 7($3)	Mem[$3+7]=$3	Stores only lowest byte
load word	lw $1, 8($2)	$1=Mem[8+$2]	Load word
load halfword	lh $1, 6($3)	$1=Mem[6+$3]	Load half; sign extend
load half unsign	lhu $1, 6($3)	$1=Mem[6+$3]	Load half; zero extend
load byte	lb $1, 5($3)	$1=Mem[5+$3]	Load byte; sign extend
load byte unsign	lbu $1, 5($3)	$1=Mem[5+$3]	Load byte; zero extend
load upper imm	lui $1,40	$1 = 40 << 16	Places immediate into upper 16 bits

MIPS Jump/Branch Instructions

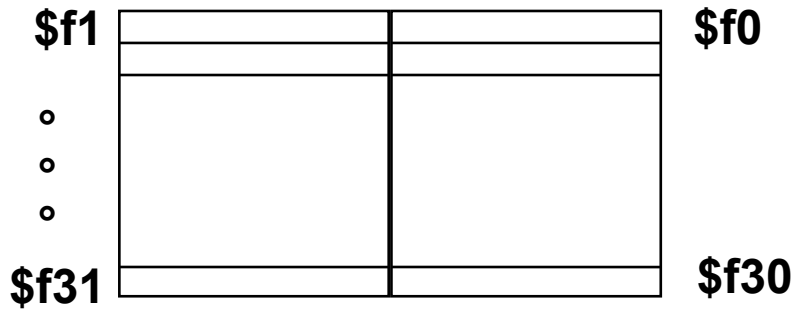
Instruction	Example	Meaning	Comments
Branch on equal	beq $s1, $s2, l	if (s1 == s2), go to L	Equal test and branch
Branch on !equal	bne $s1, $s2, l	if (s1 != s2), go to L	Not equal test and branch
jump	j 10000	PC = PC:40000	jump to address
jump register	jr $31	PC = $31	jump to address in register
jump and link	jal 10000	$31 = PC + 4;	Save PC next instruction
		PC = 40000	jump to address

Notes: Jump destinations are generally specified by a label, not an absolute address.
The destination address is word-aligned, so the instruction field is scaled by 4 to create the word address.

MIPS SLT Instructions

Instruction	Example	Meaning	Comments
set less than	slt $1,$2,$3	$1 = ($2 < $3)	compare signed <
set less than imm	slti $1,$2,100	$1 = ($2 < 100)	compare signed < constant
set less than uns	sltu $1,$2,$3	$1 = ($2 < $3)	compare unsigned <
set l. t. imm. uns.	sltiu $1,$2,100	$1 = ($2 < 100)	compare unsigned < constant

FP Registers



FP registers are paired for double-precision. Specify the even register, which holds the less-significant word.

MIPS FP Arithmetic

Instruction	Example	Meaning	Comments
FP add single	add.s $f2, $f4, $f6	$f2 = $f4 + $f6	Add single precision
FP add double	add.d $f2, $f4, $f6	$f2 = $f4 + $f6	Add double precision
FP subtract single	sub.s $f2, $f4, $f6	$f2 = $f4 - $f6	Subtract single precision
FP subtract double	sub.d $f2, $f4, $f6	$f2 = $f4 - $f6	Subtract double precision
FP multiply single	mul.s $f2, $f4, $f6	$f2 = $f4*$f6	Multiply single precision
FP multiply double	mul.d $f2, $f4, $f6	$f2 = $f4*$f6	Multiply double precision
FP divide single	div.s $f2, $f4, $f6	$f2 = $f4/$f6	Divide single precision
FP divide double	div.d $f2, $f4, $f6	$f2 = $f4/$f6	Divide double precision

MIPS FP Load/Store

Instruction	Example	Meaning
Load FP single	lwc1 $f1, 100($s2)	$f1 = Mem[$s2+100]
Store FP single	swc1 $fl, 100($s2)	Mem[$s2+100] = $f1
Load FP double	ldc1 $f0, 100($s2)	$f0 = Mem[$s2+100], $f1 = Mem[$s2+104]
Store FP double	sdc1 $f0, 100($s2)	Mem[$s2+100] = $f0 Mem[$s2+104] = $f1

MIPS FP Conditional Branch

Instruction	Example	Meaning
FP Compare single	c.lt.s $f1, $f2	if ($f1 < $f2) cond = 1 else cond = 0
FP Compare double	c.lt.d $f2, $f4	if ($f2 < $f4) cond = 1 else cond = 0
FP Branch if True	bc1t 25	if (cond == 1) go to PC + 4 + 100
FP Branch if False	bc1f 25	if (cond == 0) go to PC + 4 + 100

Comparison Instructions: It may be replaced with eq, ne, le, gt, ge

MIPS-64

- Extension of MIPS-32
- Specifies 32 64-bit registers
- Instructions remain 32-bits wide

MIPS-64

• Load 64-bit Integer	LD	
• Store 64-bit Integer		SD
• Add 64-bit integers	DADD	
• Subtract 64-bit integers		DSUB
• Mult. 64-bit integers		DMULT
• Divide 64-bit integers		DDIV
• 64-bit shifts		DSLL, DSRL